

Transport optimal pour la génération de séries temporelles financières

ADMANE Tania et EBETTY Botheinetou

2 mai 2026

1 Introduction

La génération de séries temporelles financières est un problème difficile, car les données de marché ne sont pas simplement bruitées. Elles présentent des queues épaisses, des périodes de forte volatilité, des dépendances entre actifs et des corrélations qui changent dans le temps. Un bon modèle génératif ne doit donc pas seulement reproduire la moyenne et la variance des rendements, mais aussi la structure de dépendance entre les actifs et le comportement des événements extrêmes.

Dans ce projet, nous nous intéressons au papier *Robust time series generation via Schrödinger Bridge*, qui propose une méthode de génération de séries temporelles basée sur le problème de Schrödinger Bridge. L'idée principale est de construire une dynamique stochastique contrôlée permettant de générer de nouvelles trajectoires ayant une distribution proche de celle des données réelles.

Nous avons d'abord résumé la méthode proposée dans le papier, puis nous l'avons appliquée à des données financières du S&P 500. Pour compléter l'étude, nous avons aussi implémenté deux approches alternatives liées au transport optimal : le *Flow Matching* et un modèle basé sur des *Input Convex Neural Networks* (ICNN). L'objectif est de comparer ces méthodes sur plusieurs critères : fidélité des distributions marginales, reproduction des corrélations entre actifs, dépendance temporelle, queues de distribution et capacité à produire des séries difficiles à distinguer des vraies.

2 Résumé du papier

La méthode Schrödinger Bridge Time Series (SBTS) est basée sur un problème de transport optimal entropique entre une mesure de référence et la distribution cible. Autrement dit, le but est de retrouver les distributions marginales de la vraie distribution des séries temporelles financières. La dynamique est régie par une EDS (équation différentielle stochastique) avec un drift estimé par noyau.

On dispose d'une grille temporelle $t_1, \dots, t_N$. On veut générer de nouvelles séries ayant la même distribution jointe que les données réelles.

Le problème se réécrit comme la minimisation d'une énergie de contrôle :

$$\min_{\alpha} E_P \left[\int_0^T \|\alpha_t\|^2 dt \right]$$

sous les contraintes dynamiques et marginales :

$$\begin{cases} dX_t = \alpha_t dt + dW_t^P, & X_0 = 0 \\ (X_{t_1}, \dots, X_{t_N}) \sim P_{\mu} \end{cases}$$

où :

- α_t est un drift (contrôle) à déterminer,
- W_t^P est un mouvement brownien sous P ,
- μ est la distribution jointe réelle observée aux instants $t_1, \dots, t_N$.

La solution est une EDS contrôlée :

$$dX_t = \alpha^*(t, X_t; X_{\eta(t)}) dt + dW_t^P$$

avec

$$\eta(t) = \max\{t_i : t_i \leq t\}$$

(dernier temps discret avant t).

Le drift optimal α^* pour $t \in [t_i, t_{i+1}[$ est donné par une formule faisant intervenir un noyau exponentiel F_i . Ce drift corrige la trajectoire pour qu'elle arrive à la bonne valeur $X_{t_{i+1}}$ avec la bonne probabilité conditionnelle.

Afin d'éviter que l'estimation par noyau ne devienne instable pour les grands i , les auteurs supposent que le processus est markovien d'ordre k . Cela signifie que la génération de $X_{t_{i+1}}$ ne dépend que des k dernières observations.

Cette hypothèse permet de :

- stabiliser l'estimation du drift,
- éviter la dégénérescence du noyau,
- accélérer la génération,
- réduire les risques de surapprentissage.

Le paramètre k est choisi par grid search, conjointement avec la largeur de bande h , en minimisant une erreur de prédiction sur un ensemble de validation.

2.1 Résultats

Les auteurs testent cette méthode sur plusieurs jeux de données : Stock (données financières Google), Air (données de capteurs de gaz), Sine (sinusoïdes multivariées), AR (modèle autorégressif gaussien), fBM (mouvement brownien fractionnaire).

Dans l'ensemble, SBTS surpasse ou égale les GANs qui sont pourtant l'état de l'art sur ces tâches. La seule exception est le jeu Energy, où les performances sont moins bonnes à cause de sauts brusques difficiles à modéliser avec une EDS.

2.2 Métriques utilisées

- **Discriminative score** : on entraîne un classifieur à distinguer les séries réelles des séries générées. On mesure $|acc - 0.5|$. Plus le score est proche de 0, mieux c'est. SBTS atteint 0.010 sur Stock.
- **Predictive score** : on entraîne un modèle prédictif sur les données synthétiques, puis on le teste sur les données réelles. On mesure la MAE. Plus le score est faible, mieux c'est. SBTS atteint 0.017 sur Stock.
- **Auto-corrélation / corrélation croisée** : elles mesurent la fidélité des dépendances temporelles.

3 Implémentations

3.1 Selection de data

Pour notre implémentation, nous avons choisi d'utiliser 10 actions daily du SP 500 de 2013 à 2026, issues de 7 secteurs différents : la technologie (AAPL, MSFT, GOOGL, META), les semi-conducteurs (NVDA), le e-commerce (AMZN), l'automobile (TSLA), la finance (BRK-B) et la santé (JNJ). Ce choix permet au modèle d'apprendre des dynamiques variées et des corrélations hétérogènes entre secteurs. Nous avons trouvé cette sélection intéressante aussi car l'un des enjeux majeurs en finance est la gestion du risque de portefeuille. Pour cela, il est essentiel qu'un modèle génératif reproduise fidèlement non seulement les autocorrélations propres à chaque actif, mais aussi les corrélations croisées entre actifs (par exemple, la tendance de deux actions à évoluer ensemble). Et on sait que ces actions présentent des corrélations fortes entre certaines d'entre elles (AAPL, MSFT, NVDA, AMZN), des corrélations plus faibles avec JNJ (valeur défensive), ainsi que des comportements particuliers, comme TSLA très volatile ou BRK-B moins corrélé à la technologie pure. De plus, un modèle qui fonctionne bien sur un seul stock peut échouer sur plusieurs en raison de la malédiction de la dimension ou de la difficulté à apprendre une matrice de covariance réaliste. Tester sur plusieurs stocks permet de vérifier la robustesse de l'approche SBTS dans un cadre plus réaliste et plus difficile. Enfin, en finance

quantitative, on ne trade jamais un seul actif : les stratégies (market neutral, pairs trading, hedging) impliquent plusieurs actifs.

3.2 Modeles

Afin d'appliquer les modèles et de les comparer de façon claire, nous avons effectué le *scaling* décrit dans le papier pour chaque modèle.

En effet, si les données réelles ont une variance très différente de celle attendue par le modèle, le drift α^* n'arrive pas à bien corriger la volatilité des trajectoires générées.

La solution proposée par les auteurs consiste à *rescaler* les log-rendements (au lieu de modifier Δt , ce qui est déconseillé) :

$$\tilde{R} = R \times \frac{\Delta t}{\sigma(R)}$$

où $\sigma(R)$ est l'écart-type empirique des données.

Pour revenir à l'échelle originale, on multiplie les log-rendements générés par :

$$\frac{\sigma(R)}{\Delta t}$$

Cette transformation aligne la variance des données avec celle attendue par le modèle, ce qui améliore la stabilité et le réalisme des séries générées.

3.2.1 Schrödinger Bridge Time Series

Nous avons appliqué la même méthode SBTS que les auteurs du papier. Pour cela, nous avons testé deux versions du modèle : la version non-markovienne et la version markovienne (décrite précédemment), afin de déterminer celle qui fonctionnait le mieux sur nos données.

Nous avons optimisé les hyperparamètres par (*grid search*) : pour la version non-markovienne, nous avons optimisé la largeur de bande h , et pour la version markovienne, nous avons optimisé conjointement h et l'ordre de Markov K .

Le noyau utilisé est un noyau quartic (*biweight*) à support compact : plus x est proche, plus le poids est élevé.

La méthode de génération est la suivante :

- À chaque intervalle $[t_i, t_{i+1}]$, on estime le drift à partir des M trajectoires réelles de référence.
- L'estimation utilise les poids issus du noyau ainsi qu'un terme exponentiel F_i .
- On simule ensuite l'EDS avec un schéma d'Euler à N_π sous-pas.
- Si l'on applique l'hypothèse markovienne, on ne conserve que les K dernières observations. Cela permet d'éviter que les poids deviennent nuls pour les séries longues.

3.2.2 Flow Matching

En complément de l'approche SB du papier, nous avons choisi de tester la méthode du Flow Matching. Notre objectif est de générer de nouvelles fenêtres de log-rendements (10 actions $\times$ 10 jours) qui suivent la même distribution que les données réelles.

Pour cela, le *Flow Matching* construit un flot qui transforme un bruit gaussien

$$X_0 \sim \mathcal{N}(0, I_d)$$

en la distribution des données réelles

$$X_1 \sim q.$$

Ce flot est défini par une équation différentielle ordinaire (ODE) :

$$\frac{d}{dt}X_t = v_t(X_t),$$

où v_t est un champ de vecteurs dépendant du temps qui indique comment chaque particule doit se déplacer. Ce champ v_t est appris par un réseau de neurones.

Cependant, apprendre directement v_t est trop complexe. D’après l’état de l’art, on définit donc un chemin conditionnel simple qui relie un bruit x_0 à un échantillon réel x_1 :

$$X_t = (1 - (1 - \sigma_{\min})t)X_0 + tX_1.$$

On a alors :

- Lorsque $t = 0$: X_0 est un bruit gaussien,
- Lorsque $t = 1$: X_1 est une donnée réelle (légèrement bruitée par $\sigma_{\min}$ pour régulariser).

C’est un chemin rectiligne (transport optimal) entre le bruit et la donnée réelle.

À chaque itération de l’entraînement, on procède comme suit :

- On tire un batch de fenêtres réelles x_1 .
- On génère un bruit gaussien $x_0 \sim \mathcal{N}(0, I)$.
- On échantillonne un temps $t \sim \mathcal{U}[0, 1]$.
- On construit un point intermédiaire le long du chemin :

$$X_t = \psi_t(x_0, x_1).$$

- On définit la cible :

$$u = x_1 - (1 - \sigma_{\min})x_0.$$

- Le réseau de neurones v_θ est alors entraîné à prédire u à partir de (X_t, t) .

La fonction de perte s’écrit :

$$L = E \left[\left\| v_\theta(X_t, t) - (x_1 - (1 - \sigma_{\min})x_0) \right\|^2 \right].$$

Il s’agit simplement d’une régression de type MSE. Notamment, aucune simulation d’équation différentielle n’est nécessaire pendant l’entraînement. Une fois le modèle entraîné, on peut générer de nouvelles fenêtres de la manière suivante :

- On initialise avec un bruit gaussien $X_0 \sim \mathcal{N}(0, I)$.
- On résout ensuite l’ODE de $t = 0$ à $t = 1$:

$$\frac{dX_t}{dt} = v_\theta(X_t, t), \quad X_0 \text{ donné.}$$

La valeur finale X_1 correspond alors à une nouvelle fenêtre générée.

Nous avons choisi de tester ce modèle pour plusieurs raisons. D’une part, la génération est nettement plus rapide lorsque l’on souhaite produire un grand nombre de fenêtres. D’autre part, aucune estimation par noyau n’est nécessaire, ce qui simplifie considérablement la mise en œuvre. Comme discuté précédemment, les méthodes à noyau peuvent poser problème pour les séries longues, ce qui conduit à introduire une hypothèse de Markov. Toutefois, cette approximation entraîne une perte d’information. Le *Flow Matching* ne souffre pas de cette limitation. Enfin, ce modèle ne requiert pas de réglage d’hyperparamètres sensibles tels que la largeur de bande h , dont nous avons vu qu’elle peut fortement influencer les résultats.

3.2.3 ICNN

En plus de SBTS et du Flow Matching, nous avons choisi de tester une approche basée sur les *Input Convex Neural Networks* (ICNN). L’objectif est d’avoir une troisième méthode issue du transport optimal, mais avec une formulation différente. Ici, on ne simule pas une EDS comme dans SBTS, et on n’apprend pas non plus un champ de vecteurs dépendant du temps comme dans Flow Matching. L’ICNN apprend plutôt une carte de transport déterministe entre une distribution source simple et la distribution des données réelles.

Comme précédemment, les données utilisées sont des fenêtres de log-rendements de taille 10×9 , c’est-à-dire 10 jours et 9 actifs. Avant d’être données au modèle, ces fenêtres sont aplaties sous forme de vecteurs :

$$x \in \mathbb{R}^D, \quad D = 10 \times 9 = 90.$$

L’idée vient de la formulation de Monge du transport optimal. On apprend un potentiel convexe

$$\varphi_\theta : \mathbb{R}^D \rightarrow \mathbb{R},$$

et la carte de transport est donnée par le gradient de ce potentiel :

$$T_\theta(z) = \nabla \varphi_\theta(z).$$

Ainsi, pour générer une nouvelle fenêtre, on part d'un bruit z , puis on applique la carte T_θ . Contrairement à un réseau classique, le réseau ne prédit donc pas directement la sortie générée mais il apprend un potentiel convexe, et la sortie correspond au gradient de ce potentiel.

Pour la distribution source, nous avons utilisé une loi de Student :

$$z \sim t_\nu, \quad \nu = 5.$$

Ce choix est plus adapté que le bruit gaussien standard dans notre cas, car les rendements financiers ont souvent des queues plus épaisses qu'une loi normale. Le bruit est ensuite normalisé afin de garder une échelle stable pendant l'entraînement.

La convexité du potentiel est imposée par la structure de l'ICNN. Les poids entre couches cachées sont contraints à être positifs, et les activations utilisées sont convexes et croissantes. Cela permet de garantir que la fonction apprise reste convexe par rapport à l'entrée.

Pour l'initialisation, nous avons utilisé une carte de Brenier gaussienne. L'idée est de ne pas commencer avec une carte totalement arbitraire, mais avec une transformation qui prend déjà en compte la covariance empirique des données. Cela est important dans notre cas, car les corrélations entre actifs sont une partie centrale du problème.

La fonction de perte combine plusieurs termes. Nous avons utilisé une divergence de Sinkhorn, une perte de Wasserstein slicé et une perte MMD multi-échelle. À cela, nous avons ajouté des pénalités sur les statistiques importantes pour les séries financières, comme les moyennes, les écarts-types, les covariances, les corrélations entre actifs, les autocovariances et les queues de distribution.

La loss peut s'écrire de manière simplifiée :

$$\mathcal{L}(\theta) = \lambda_{\text{OT}} \mathcal{L}_{\text{Sinkhorn}} + \lambda_{\text{SW}} \mathcal{L}_{\text{SW}} + \lambda_{\text{MMD}} \mathcal{L}_{\text{MMD}} + \lambda_{\text{stat}} \mathcal{L}_{\text{stat}} + \lambda_{\text{tail}} \mathcal{L}_{\text{tail}}.$$

Une fois le modèle entraîné, la génération se fait directement :

$$\hat{x} = T_\theta(z) = \nabla \varphi_\theta(z).$$

Le vecteur obtenu est ensuite remis sous forme de fenêtre de taille (10, 9).

4 Résultats

4.0.1 Schrödinger Bridge Time Series

Nous avons testé les deux variantes du modèle, avec et sans hypothèse markovienne, et observé que la version non-markovienne donne les meilleurs résultats.

Cela s'explique par la structure des données : l'autocorrélation au lag 1 de nos neuf actifs varie entre -0.114 et -0.008 , comme indiqué dans le Tableau 1, ce qui est cohérent avec l'hypothèse d'efficience faible des marchés.

Ces valeurs étant très proches de zéro, le conditionnement markovien n'apporte pas de gain significatif. Au contraire, la restriction de l'information à un nombre limité d'observations passées ($K \geq 1$) induit un rétrécissement du noyau, ce qui augmente la variance de l'estimation du drift davantage qu'il ne réduit le biais.

Actif	ACF(1)
AAPL	-0.040060
MSFT	-0.019710
NVDA	-0.105793
TSLA	-0.042265
AMZN	-0.075124
GOOGL	-0.032152
META	-0.113857
BRK-B	-0.072007
JNJ	-0.007853

TABLE 1 – Autocorrélation au lag 1 (ACF(1)) pour différents actifs

Sur les neuf actifs du S&P 500, SBTS obtient de très bons scores, comme les résultats du papier. Que ce soit pour les moyennes, les écarts-types, les quantiles 1%/99%, et même les valeurs extrêmes (min/max), ils sont préservés à environ 1% près sur l'ensemble des actifs. Le score discriminatif est de 0.0185 ± 0.0066 , ce qui est nettement meilleur que les approches GAN, comme le dit le papier.

La métrique la plus marquante concerne la persistance de volatilité mesurée via l'autocorrélation des rendements en valeur absolue. Cette métrique mesure le *volatility clustering*, qui est observé sur les marchés financiers et qui stipule que les périodes agitées succèdent aux périodes agitées, et les périodes calmes succèdent aux périodes calmes. Avec un score d'écart de 0.00520, SBTS reproduit très bien ce phénomène.

La cross-correlation mesure le fait que la structure de corrélation entre les actifs est bien respectée. Le score de 0.0211 signifie que les corrélations synthétiques s'écartent en moyenne de seulement 0.02 des valeurs réelles, ce qui est très satisfaisant, surtout étant donné que certains actifs sont très corrélés entre eux.

Feature	1% Data	1% SBTS	99% Data	99% SBTS	Mean Data	Mean SBTS	Std Data	Std SBTS
AAPL	-0.049	-0.050	0.047	0.048	0.001	0.001	0.018	0.018
MSFT	-0.058	-0.056	0.056	0.053	0.001	0.001	0.020	0.020
NVDA	-0.030	-0.032	0.031	0.029	0.001	0.000	0.012	0.011
TSLA	-0.048	-0.049	0.048	0.044	0.001	0.001	0.018	0.017
AMZN	-0.029	-0.031	0.028	0.028	0.000	0.000	0.011	0.011
GOOGL	-0.060	-0.061	0.060	0.058	0.001	0.001	0.024	0.023
META	-0.043	-0.044	0.045	0.043	0.001	0.001	0.017	0.016
BRK-B	-0.075	-0.072	0.073	0.072	0.002	0.002	0.029	0.028
JNJ	-0.095	-0.095	0.106	0.099	0.002	0.001	0.036	0.035

TABLE 2 – Comparaison des statistiques descriptives entre données réelles et SBTS

Metric	SBTS
Discriminative	0.0185 ± 0.007
Predictive	0.0254 ± 0.000
Auto-corr (abs/vol)	0.0052
Cross-corr	0.0211

TABLE 3 – Performance de SBTS sur différentes métriques

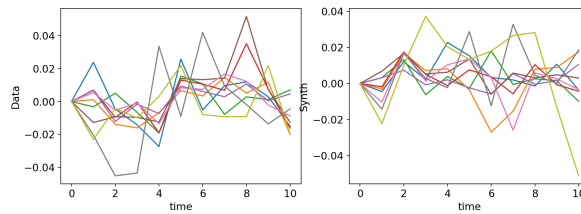


FIGURE 1 – Comparaison des trajectoires de log-returns réelles et générées par SBTS

4.0.2 Flow Matching

Flow Matching reproduit fidèlement les moments centraux et la dispersion des distributions empiriques : les moyennes et écarts-types coïncident avec les valeurs réelles à la troisième décimale près sur l'ensemble des neuf actifs. Les quantiles 1% et 99% sont également bien préservés, avec un écart maximal de 0.014 sur JNJ (0.106 réel contre 0.092 synthétique au 99^e centile).

Cependant, contrairement à SBTS, FM tend à comprimer systématiquement les queues de distribution. Sur AAPL, le quantile 1% passe de -0.049 à -0.045 ; sur MSFT, de -0.058 à -0.053 ; sur BRK-B, de -0.075 à -0.069 ; sur JNJ, le quantile 99% passe de 0.106 à 0.092. Cet effet de rétrécissement, bien que modéré, est cohérent à travers tous les actifs et reflète une caractéristique structurelle des modèles génératifs paramétriques : le réseau de vitesse appris pousse l'écoulement ODE vers les régions de forte

densité observée, ce qui induit un biais vers le centre de la distribution et une sous-représentation systématique des événements rares.

Le score discriminatif de 0.0230 ± 0.007 obtenu par FM est très proche de celui de SBTS.

En revanche, FM révèle une faiblesse plus marquée sur les métriques de dépendance temporelle. L'autocorrélation des rendements absolus atteint 0.0115, soit plus du double de celle de SBTS (0.0052), ce qui traduit une reproduction moins fidèle de la persistance de volatilité. De même, le score de cross-corrélation de 0.0307, environ 50% plus élevé que celui de SBTS (0.0211), confirme une difficulté à reproduire fidèlement les dépendances entre actifs.

Flow Matching, dans sa formulation originale d'optimal transport, démontre ainsi de très bonnes performances en termes de fidélité globale (scores discriminatif et prédictif proches du state-of-the-art), mais présente des limites identifiées sur les métriques structurelles spécifiques aux séries financières.

Feature	1% Data	1% FM	99% Data	99% FM	Mean Data	Mean FM	Std Data	Std FM
AAPL	-0.049	-0.045	0.047	0.045	0.001	0.001	0.018	0.018
MSFT	-0.058	-0.053	0.056	0.052	0.001	0.001	0.020	0.020
NVDA	-0.030	-0.029	0.031	0.028	0.001	0.000	0.012	0.011
TSLA	-0.048	-0.044	0.048	0.044	0.001	0.001	0.018	0.017
AMZN	-0.029	-0.027	0.028	0.028	0.000	0.000	0.011	0.011
GOOGL	-0.060	-0.058	0.060	0.059	0.001	0.001	0.024	0.023
META	-0.043	-0.041	0.045	0.042	0.001	0.001	0.017	0.016
BRK-B	-0.075	-0.069	0.073	0.072	0.002	0.002	0.029	0.028
JNJ	-0.095	-0.087	0.106	0.092	0.002	0.002	0.036	0.036

TABLE 4 – Comparaison des statistiques descriptives entre données réelles et FM

Metric	FM
Discriminative	0.0230 ± 0.007
Predictive	0.0254 ± 0.000
Auto-corr (abs/vol)	0.0115
Cross-corr	0.0307

TABLE 5 – Performance de FM sur différentes métriques

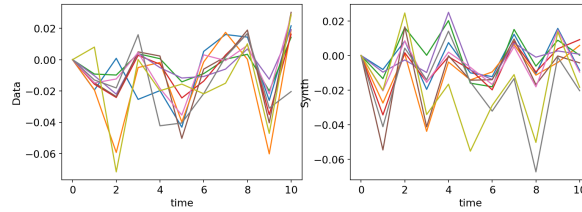


FIGURE 2 – Comparaison des trajectoires de log-returns réelles et générées par FM

4.0.3 ICNN

L'ICNN reproduit assez bien les statistiques marginales des données. Les moyennes et les écarts-types restent proches des vraies valeurs pour presque tous les actifs. Les quantiles à 1% et 99% sont aussi raisonnablement préservés, même si l'on observe une légère compression des queues pour certains actifs.

Feature	1% Data	1% ICNN	99% Data	99% ICNN	Mean Data	Mean ICNN	Std Data	Std ICNN
AAPL	-0.049	-0.046	0.047	0.046	0.001	-0.000	0.018	0.019
MSFT	-0.058	-0.053	0.056	0.052	0.001	0.000	0.020	0.022
NVDA	-0.030	-0.030	0.031	0.028	0.001	-0.000	0.012	0.012
TSLA	-0.048	-0.045	0.048	0.045	0.001	0.000	0.018	0.019
AMZN	-0.029	-0.028	0.028	0.029	0.000	-0.000	0.011	0.012
GOOGL	-0.060	-0.057	0.060	0.059	0.001	0.001	0.024	0.024
META	-0.043	-0.042	0.045	0.043	0.001	0.000	0.017	0.018
BRK-B	-0.075	-0.070	0.073	0.075	0.002	0.002	0.029	0.030
JNJ	-0.095	-0.090	0.106	0.094	0.002	0.002	0.036	0.039

TABLE 6 – Comparaison des statistiques descriptives entre données réelles et ICNN

On voit que les écarts-types générés sont proches des écarts-types réels. Par exemple, pour AAPL, on passe de 0.018 à 0.019, pour GOOGL de 0.024 à 0.024, et pour BRK-B de 0.029 à 0.030. Les moyennes sont également très proches de zéro, comme dans les données réelles.

Cependant, l'ICNN a plus de difficulté à reproduire les mouvements extrêmes. Par exemple, pour GOOGL, le maximum réel est beaucoup plus grand que ce que le modèle génère. Le même phénomène apparaît aussi pour plusieurs actifs. Cela montre que l'ICNN capture bien le centre de la distribution, mais qu'il a tendance à réduire les événements rares.

Les métriques globales confirment ce comportement. Le score prédictif est proche de ceux obtenus par SBTS et Flow Matching, ce qui signifie que les données générées gardent une structure utile pour la prédiction. En revanche, le score discriminatif est moins bon, donc les séries générées par ICNN sont plus faciles à distinguer des vraies séries.

Metric	ICNN
Discriminative	0.0980 ± 0.037
Predictive	0.0261 ± 0.000
Auto-corr (raw)	0.0160
Auto-corr (abs/vol)	0.0128
Cross-corr	0.0606

TABLE 7 – Performance de ICNN sur différentes métriques

La différence la plus importante apparaît sur la cross-correlation. Le score de l'ICNN est de 0.0606, ce qui est plus élevé que SBTS et Flow Matching. Cela indique que le modèle reproduit moins bien les corrélations entre actifs.

L'autocorrélation des rendements absolus vaut 0.0128. Cette métrique mesure le *volatility clustering*. Le score reste correct, mais il est moins bon que celui de SBTS. Cela peut s'expliquer par le fait que l'ICNN travaille sur des fenêtres aplaties et apprend une carte statique. Il ne modélise donc pas directement la dynamique temporelle comme SBTS.

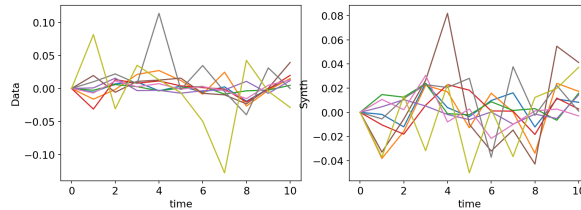


FIGURE 3 – Comparaison des trajectoires de log-returns réelles et générées par ICNN

5 Conclusion

Dans ce projet, nous avons étudié la génération de séries temporelles financières avec des méthodes liées au transport optimal. Nous avons commencé par présenter la méthode SBTS du papier, puis nous

l'avons comparée à deux autres approches : Flow Matching et ICNN.

Sur nos données, SBTS donne les meilleurs résultats globaux. La méthode reproduit bien les statistiques marginales, les queues de distribution, les corrélations entre actifs et la persistance de volatilité. La version non-markovienne fonctionne mieux que la version markovienne, ce qui est cohérent avec les faibles autocorrélations observées sur nos rendements.

Flow Matching donne aussi de bons résultats, surtout sur les moyennes, les écarts-types et le score discriminatif. Son avantage principal est qu'il est simple à entraîner et rapide pour générer de nouvelles trajectoires. En revanche, il compresse davantage les queues de distribution et reproduit moins bien certaines dépendances temporelles.

L'ICNN donne une baseline intéressante de transport optimal statique. Le modèle reproduit correctement les statistiques marginales et obtient un score prédictif proche des autres méthodes. Cependant, il est moins performant sur le score discriminatif, la cross-correlation et le *volatility clustering*. Cela vient probablement du fait qu'il apprend une carte déterministe sur des fenêtres aplaties, sans modéliser explicitement la dynamique temporelle.

Au final, SBTS semble être la méthode la plus adaptée dans notre cas, car elle conserve mieux les propriétés importantes des séries financières. Flow Matching reste une alternative efficace et plus simple à utiliser. ICNN permet de tester une approche de transport optimal statique, mais il est moins adapté pour reproduire les dépendances complexes et les mouvements extrêmes du marché.

Références

- [1] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel et M. Le. *Flow Matching for Generative Modeling*. ICLR, 2023. <https://openreview.net/forum?id=PqvMRDCJT9t>
- [2] A. Alouadi, B. Barreau, L. Carlier et H. Pham. *Robust time series generation via Schrödinger Bridge : a comprehensive evaluation*. <https://arxiv.org/abs/2503.02943>
- [3] A. V. Makkuva, A. Taghvaei, S. Oh et J. D. Lee. *Optimal Transport Mapping via Input Convex Neural Networks*. Proceedings of the 37th International Conference on Machine Learning, PMLR, 2020. <https://proceedings.mlr.press/v119/makkuva20a.html>